

EASY GUIDE — Game of Active Directory (GOAD) — Windows Installation Setup

 medium.com/@shanksf/game-of-active-directory-goad-windows-installation-setup-5d987f0228bd

Shanks

April 26, 2024

Press enter or click to view image in full size



- [PREREQUISITES](#)
- [STEPS](#)
 - [Install VMware Workstation](#)
 - [Install Vagrant for Windows](#)
 - [Install Vagrant VMware Utility](#)

- [TROUBLESHOOTING TIPS](#)
- [BONUS: ELK](#)

Main project by Mayfly: <https://github.com/Orange-Cyberdefense/GOAD>

This was fully tested only on a Windows 10 machine with 64GB of RAM.

This may also work for GOAD-light and NHA.

It's supposed to be the easiest guide on the internet for using and appreciating GOAD.

Hope it works for you.

PREREQUISITES

- Windows 10 (HOST) • VMWARE Workstation (Also may work with Virtualbox, just adapt) • Vagrant (Windows) • Kali or Ubuntu (Choose any distro you want) • A lot of disk space (~120GB) • A lot of ram (At least 32GB for the main GOAD)

STEPS

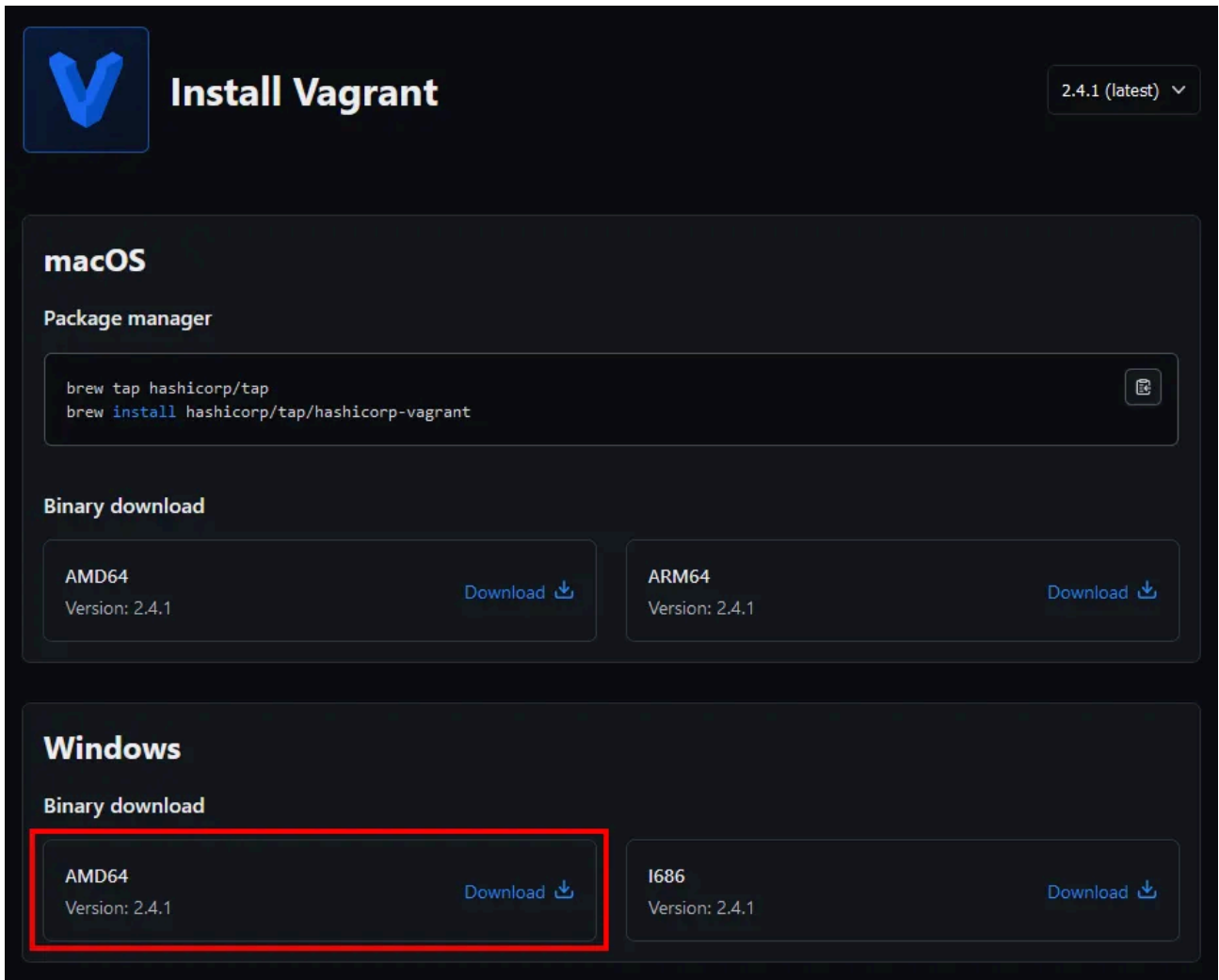
Install VMware Workstation

<https://www.vmware.com/products/workstation-pro/workstation-pro-evaluation.html>

Install Vagrant for Windows

https://developer.hashicorp.com/vagrant/install?product_intent=vagrant#Windows

Press enter or click to view image in full size



The screenshot shows the 'Install Vagrant' page with a dark theme. At the top left is the Vagrant logo (a blue 'V' in a square). To its right is the title 'Install Vagrant' in white. Further right is a dropdown menu showing '2.4.1 (latest)' with a downward arrow. Below this is a section for 'macOS'. Under 'macOS', there is a 'Package manager' section with a code block containing two lines: 'brew tap hashicorp/tap' and 'brew install hashicorp/tap/hashicorp-vagrant'. To the right of the code is a copy icon. Below the package manager is a 'Binary download' section with two buttons: 'AMD64 Version: 2.4.1' and 'ARM64 Version: 2.4.1', each with a 'Download' link and a download icon. Below this is a section for 'Windows'. Under 'Windows', there is a 'Binary download' section with two buttons: 'AMD64 Version: 2.4.1' and 'I686 Version: 2.4.1', each with a 'Download' link and a download icon. The 'AMD64' button under Windows is highlighted with a red rectangular border.

Install Vagrant 2.4.1 (latest) ▾

macOS

Package manager

```
brew tap hashicorp/tap  
brew install hashicorp/tap/hashicorp-vagrant
```

Binary download

Architecture	Version	Download
AMD64	2.4.1	Download ⬇
ARM64	2.4.1	Download ⬇

Windows

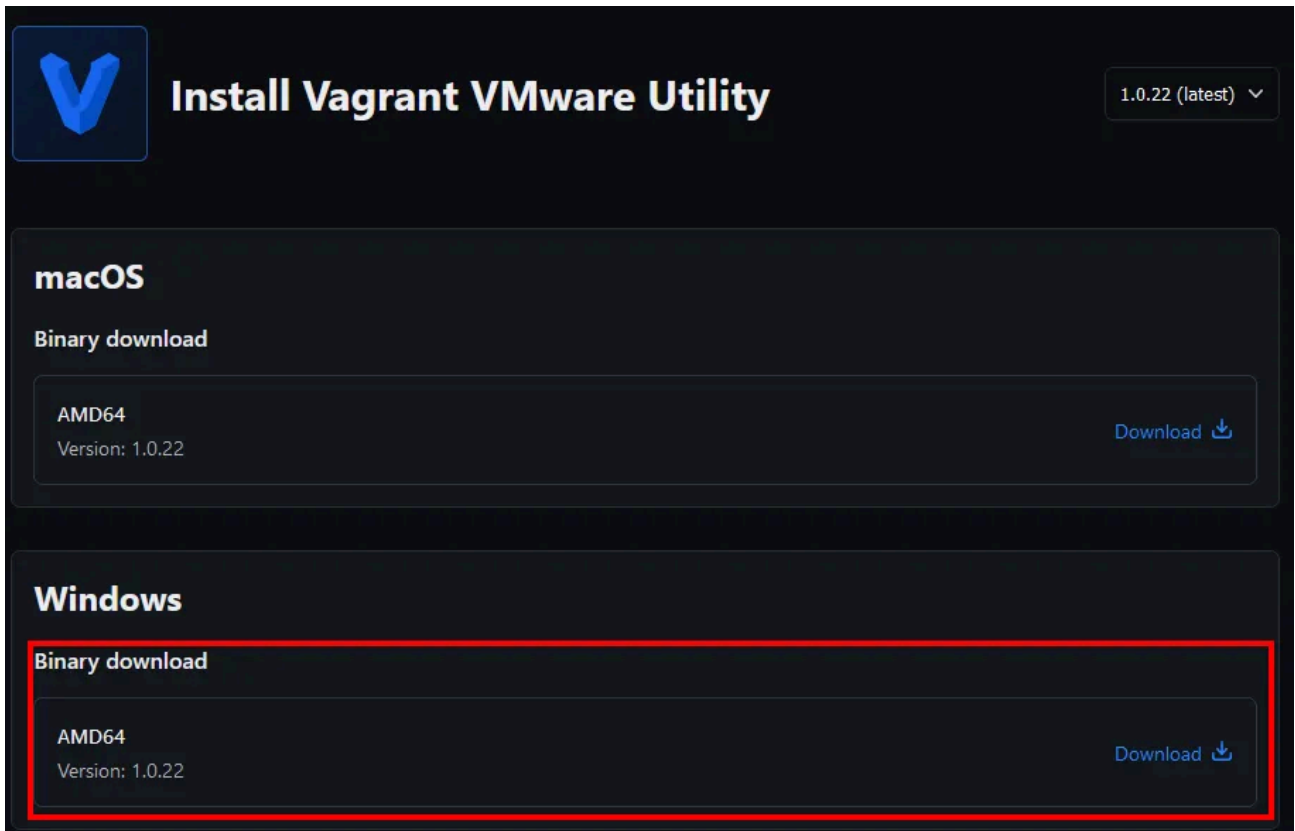
Binary download

Architecture	Version	Download
AMD64	2.4.1	Download ⬇
I686	2.4.1	Download ⬇

Install Vagrant VMware Utility

<https://developer.hashicorp.com/vagrant/install/vmware>

Press enter or click to view image in full size



In the same folder your vagrant.exe is (default **C:\Program Files\Vagrant\bin**), open a new cmd and type:

```
vagrant plugin install vagrant-vmware-desktop
```

```
C:\Program Files\Vagrant\bin>vagrant plugin install vagrant-vmware-desktop
Installing the 'vagrant-vmware-desktop' plugin. This can take a few minutes...
Installed the plugin 'vagrant-vmware-desktop (3.0.3)'!

C:\Program Files\Vagrant\bin>_
```

Clone GOAD to a folder of your liking (recommend being the biggest hard drive you have and not C:) or just Download the zip in their github <https://github.com/Orange-Cyberdefense/GOAD>

Go to providers folder (cd GOAD/ad/GOAD/providers/vmware) and type

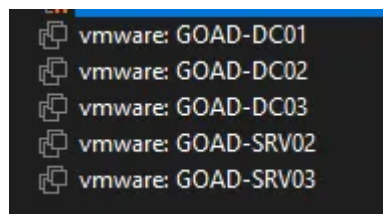
```
vagrant up
```

Press enter or click to view image in full size

```
Microsoft Windows [Version 10.0.19044.4291]
(c) Microsoft Corporation. All rights reserved.

E:\GOAD-main\ad\GOAD\providers\vmware>vagrant up
Bringing machine 'GOAD-DC01' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-DC02' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-DC03' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-SRV02' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-SRV03' up with 'vmware_desktop' provider...
==> GOAD-DC01: Cloning VMware VM: 'StefanScherer/windows_2019'. This can take some time...
==> GOAD-DC01: Checking if box 'StefanScherer/windows_2019' version '2021.05.15' is up to date...
==> GOAD-DC01: There was a problem while downloading the metadata for your box
==> GOAD-DC01: to check for updates. This is not an error, since it is usually due
==> GOAD-DC01: to temporary network problems. This is just a warning. The problem
==> GOAD-DC01: encountered was:
==> GOAD-DC01:
==> GOAD-DC01: Failed to connect to vagrantcloud.com port 443 after 234 ms: Couldn't connect to server
==> GOAD-DC01:
==> GOAD-DC01: If you want to check for box updates, verify your network connection
==> GOAD-DC01: is valid and try again.
==> GOAD-DC01: Verifying vmnet devices are healthy...
```

This will download and setup all the 5 machines necessary to your vmware. All 5 machines should start and appear on your vmware library.



If the error below appears, try disabling any virtualbox network adaptors you have. This will conflict with GOAD.

Press enter or click to view image in full size

```
==> GOAD-DC01: Verifying vmnet devices are healthy...
The host only network with the IP '192.168.56.10' would collide with
another device 'Ethernet 3'. This means that VMware cannot create
a proper networking device to route to your VM. Please choose
another IP or shut down the existing device.
```

To able to communicate with the other machines, you gotta add a **new NIC** to your vmware and add it to your Kali (or distro of your choice).

Go to VMWare Workstation's Virtual Network Editor



Add a new NIC adapter to **192.168.56.0 (Host-only)** then add to you Kali/Ubuntu and keep your NAT otherwise you won't have internet.

VMnet10	NAT	NAT	Connected	Enabled	192.168.120.0
VMnet10	Host-only	-	Connected	Enabled	192.168.56.0

Add Network... Remove Network Rename Network...

VMnet Information

☐ Bridged (connect VMs directly to the external network)
 Bridged to: Automatic Automatic Settings...

☐ NAT (shared host's IP address with VMs) NAT Settings...

☒ Host-only (connect VMs internally in a private network)

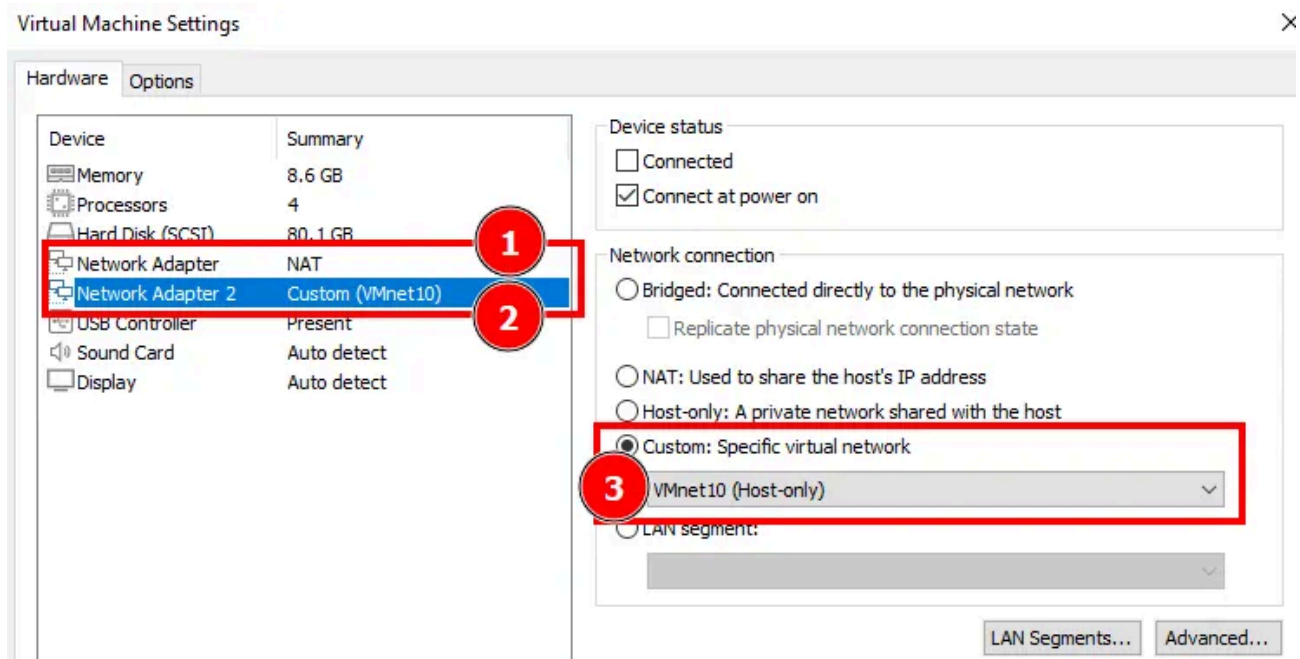
☒ Connect a host virtual adapter to this network
 Host virtual adapter name: VMware Network Adapter VMnet10

☒ Use local DHCP service to distribute IP address to VMs DHCP Settings...

Subnet IP: 192 . 168 . 56 . 0 Subnet mask: 255 . 255 . 255 . 0

Add a host only vmnet to **192.168.56.0**

Press enter or click to view image in full size



Add a new NIC on Kali for the same vmnet you set up earlier and keep your NAT

Now, inside the distro of your choice (I'm using Kali here), fire up your terminal and install some dependencies that are needed for Ansible etc.

```
pip install --upgrade pip
pip install ansible-core==2.12.6
pip install pywinrm
```

```
sudo apt install sshpass lftp rsync openssh-client
```

Clone the GOAD repo again in your Kali

Install ansible-galaxy:

```
sudo apt install ansible-core
```

Then go to your ansible folder (cd GOAD/ansible) and install the requirements needed:

```
ansible-galaxy install -r requirements.yml
```

Go back to the root folder of GOAD and run the local installation

```
./goad.sh -t install -l GOAD -p vmware -m local -a
```

Press enter or click to view image in full size

```
kali@kali: ~/GOAD
File Actions Edit View Help
(kali@kali)~[~]
$ cd GOAD

(kali@kali)~/GOAD
$ ./goad.sh -t install -l GOAD -p vmware -m local -a
[✓] Task: install
[✓] Lab: GOAD
[✓] Provider: vmware
[✓] Method: local
[✓] Run ansible only
[✓] folder ad/GOAD/providers/vmware found
[✓] Launch installation for: GOAD / vmware / local
[+] Current folder /home/kali/GOAD/ansible
[+] Current LAB : GOAD
[+] Current PROVIDER : vmware
[+] Ansible command : ansible-playbook -i ../ad/GOAD/data/inventory -i ../ad/GOAD/providers/vmware/inventory
[+] Running all the playbook to setup the lab
[+] Entering GOAD build
[+] Restart counter: 0
[✓] Running command with timeout 30min: ansible-playbook -i ../ad/GOAD/data/inventory -i ../ad/GOAD/providers/vmware/inventory build.yml

PLAY [build all] *****
[started TASK: Gathering Facts on dc01]
[started TASK: Gathering Facts on dc02]
[started TASK: Gathering Facts on dc03]
[started TASK: Gathering Facts on srv02]
```

This is gonna take a while... grab a coffee ☕ or go watch something.

When this message appear, it all went well and now you can test it.

```
[✓] Command successfully executed
[✓] your lab : GOAD is successfully setup ! have fun ;)
Build in 74 minutes and 20 seconds.
/home/kali/GOAD
```

To test the connection, you can use netexec as following

```
nxc smb 192.168.56.0/24
```

Press enter or click to view image in full size

```
(kali@kali)~/GOAD
$ nxc smb 192.168.56.0/24
SMB 192.168.56.12 445 MEEREN [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:MEEREN) (domain:essos.local) (signing:True) (SMBv1:True)
SMB 192.168.56.11 445 WINTERFELL [*] Windows 10 / Server 2019 Build 17763 x64 (name:WINTERFELL) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB 192.168.56.10 445 KINGSLANDING [*] Windows 10 / Server 2019 Build 17763 x64 (name:KINGSLANDING) (domain:sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB 192.168.56.23 445 BRAAVOS [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:BRAAVOS) (domain:essos.local) (signing:False) (SMBv1:True)
SMB 192.168.56.22 445 CASTELBLACK [*] Windows 10 / Server 2019 Build 17763 x64 (name:CASTELBLACK) (domain:north.sevenkingdoms.local) (signing:False) (SMBv1:False)
Running nxc against 256 targets 100% 0:00:00
```

Congratulations! You've now setup GOAD perfectly!

TROUBLESHOOTING TIPS

If some step on Ansible fails, you can simply destroy the vm you want (i.e: GOAD-DC01) and install again with Vagrant.

```
vagrant destroy GOAD-DC01
```

```
E:\GOAD-main\ad\GOAD\providers\vmware>vagrant destroy GOAD-DC01
GOAD-DC01: Are you sure you want to destroy the 'GOAD-DC01' VM? [y/N] Y
==> GOAD-DC01: Stopping the VMware VM...
==> GOAD-DC01: Deleting the VM...

E:\GOAD-main\ad\GOAD\providers\vmware>vagrant up
Bringing machine 'GOAD-DC01' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-DC02' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-DC03' up with 'vmware_desktop' provider...
Bringing machine 'GOAD-SRV02' up with 'vmware_desktop' provider...
```

After that, just run **vagrant up** to download the machine then the local installation step again.

If you want to stop all machines, use **vagrant halt**.

BONUS: ELK

1. Comment all boxes and uncomment ELK on Vagrantfile — location:
\ad\GOAD\providers\vmware\Vagrantfile

```
Vagrant.configure("2") do |config|
```

```
ENV['VAGRANT_DEFAULT_PROVIDER'] = 'vmware_desktop'
```

```
boxes = [
  # windows server 2022 : don't work for now
  #{ :name => "DC01", :ip => "192.168.56.10", :box =>
"StefanScherer/windows_2022", :box_version => "2021.08.23", :os => "windows"},
  # windows server 2019
  # { :name => "GOAD-DC01", :ip => "192.168.56.10", :box =>
"StefanScherer/windows_2019", :box_version => "2021.05.15", :os => "windows"},
  # windows server 2019
  #{ :name => "GOAD-DC02", :ip => "192.168.56.11", :box =>
"StefanScherer/windows_2019", :box_version => "2021.05.15", :os => "windows"},
  # windows server 2016
  #{ :name => "GOAD-DC03", :ip => "192.168.56.12", :box =>
"StefanScherer/windows_2016", :box_version => "2017.12.14", :os => "windows"},
  # windows server 2019
  #{ :name => "SRV01", :ip => "192.168.56.21", :box =>
"StefanScherer/windows_2019", :box_version => "2020.07.17", :os => "windows"},
  # windows server 2019
  # { :name => "GOAD-SRV02", :ip => "192.168.56.22", :box =>
"StefanScherer/windows_2019", :box_version => "2020.07.17", :os => "windows"},
  # windows server 2016
  #{ :name => "GOAD-SRV03", :ip => "192.168.56.23", :box =>
"StefanScherer/windows_2016", :box_version => "2019.02.14", :os => "windows"}

  # ELK
  { :name => "GOAD-ELK", :ip => "192.168.56.50", :box => "bento/ubuntu-18.04", :os
=> "linux",
    :forwarded_port => [
```

```

    { :guest => 22, :host => 2210, :id => "ssh" }
  ]
}
]

config.vm.provider "virtualbox" do |v|
  v.memory = 4000
  v.cpus = 2
end

config.vm.provider "vmware_desktop" do |v|
  v.vmx["memsize"] = "4000"
  v.vmx["numvcpus"] = "2"
  # v.force_vmware_license = "workstation" # force the licence for fix some
vagrant plugin issue
end

# disable rdp forwarded port inherited from StefanScherer box
config.vm.network :forwarded_port, guest: 3389, host: 3389, id: "rdp",
auto_correct: true, disabled: true

# no autoupdate if vagrant-vbguest is installed
if Vagrant.has_plugin?("vagrant-vbguest") then
  config.vbguest.auto_update = false
end

config.vm.boot_timeout = 600
config.vm.graceful_halt_timeout = 600
config.winrm.retry_limit = 30
config.winrm.retry_delay = 10

boxes.each do |box|
  config.vm.define box[:name] do |target|
    # BOX
    target.vm.provider "virtualbox" do |v|
      v.name = box[:name]
      v.customize ["modifyvm", :id, "--groups", "/GOAD"]
    end
    target.vm.box_download_insecure = box[:box]
    target.vm.box = box[:box]
    if box.has_key?(:box_version)
      target.vm.box_version = box[:box_version]
    end

    # issues/49
    target.vm.synced_folder '.', '/vagrant', disabled: true
  end
end

```

```

# IP
target.vm.network :private_network, ip: box[:ip]

# OS specific
if box[:os] == "windows"
  target.vm.guest = :windows
  target.vm.communicator = "winrm"
  target.vm.provision :shell, :path => "../../../../vagrant/Install-
WMF3Hotfix.ps1", privileged: false
  target.vm.provision :shell, :path =>
"../../../../vagrant/ConfigureRemotingForAnsible.ps1", privileged: false

  # fix ip for vmware
  if ENV['VAGRANT_DEFAULT_PROVIDER'] == "vmware_desktop"
    target.vm.provision :shell, :path => "../../../../vagrant/fix_ip.ps1",
privileged: false, args: box[:ip]
  end

else
  target.vm.communicator = "ssh"
end

if box.has_key?(:forwarded_port)
  # forwarded port explicit
  box[:forwarded_port] do |forwarded_port|
    target.vm.network :forwarded_port, guest: forwarded_port[:guest], host:
forwarded_port[:host], host_ip: "127.0.0.1", id: forwarded_port[:id]
  end
end

end
end
end

```

2. vagrant up to install ELK. When it finishes, revert to the original file again:

```
Vagrant.configure("2") do |config|
```

```
ENV['VAGRANT_DEFAULT_PROVIDER'] = 'vmware_desktop'
```

```

boxes = [
  # windows server 2022 : don't work for now
  #{ :name => "DC01", :ip => "192.168.56.10", :box =>
"StefanScherer/windows_2022", :box_version => "2021.08.23", :os => "windows"},
  # windows server 2019
  { :name => "GOAD-DC01", :ip => "192.168.56.10", :box =>
"StefanScherer/windows_2019", :box_version => "2021.05.15", :os => "windows"},
  # windows server 2019

```

```

    { :name => "GOAD-DC02", :ip => "192.168.56.11", :box =>
"StefanScherer/windows_2019", :box_version => "2021.05.15", :os => "windows"},
    # windows server 2016
    { :name => "GOAD-DC03", :ip => "192.168.56.12", :box =>
"StefanScherer/windows_2016", :box_version => "2017.12.14", :os => "windows"},
    # windows server 2019
    #{ :name => "SRV01", :ip => "192.168.56.21", :box =>
"StefanScherer/windows_2019", :box_version => "2020.07.17", :os => "windows"},
    # windows server 2019
    { :name => "GOAD-SRV02", :ip => "192.168.56.22", :box =>
"StefanScherer/windows_2019", :box_version => "2020.07.17", :os => "windows"},
    # windows server 2016
    { :name => "GOAD-SRV03", :ip => "192.168.56.23", :box =>
"StefanScherer/windows_2016", :box_version => "2019.02.14", :os => "windows"}

    # ELK
# { :name => "GOAD-ELK", :ip => "192.168.56.50", :box => "bento/ubuntu-18.04", :os
=> "linux",
#   :forwarded_port => [
#     {:guest => 22, :host => 2210, :id => "ssh"}
#   ]
# }
]

config.vm.provider "virtualbox" do |v|
  v.memory = 4000
  v.cpus = 2
end

config.vm.provider "vmware_desktop" do |v|
  v.vmx["memsize"] = "4000"
  v.vmx["numvcpus"] = "2"
  # v.force_vmware_license = "workstation" # force the licence for fix some
vagrant plugin issue
end

# disable rdp forwarded port inherited from StefanScherer box
config.vm.network :forwarded_port, guest: 3389, host: 3389, id: "rdp",
auto_correct: true, disabled: true

# no autoupdate if vagrant-vbguest is installed
if Vagrant.has_plugin?("vagrant-vbguest") then
  config.vbguest.auto_update = false
end

config.vm.boot_timeout = 600
config.vm.graceful_halt_timeout = 600
config.winrm.retry_limit = 30

```

```

config.winrm.retry_delay = 10

boxes.each do |box|
  config.vm.define box[:name] do |target|
    # BOX
    target.vm.provider "virtualbox" do |v|
      v.name = box[:name]
      v.customize ["modifyvm", :id, "--groups", "/GOAD"]
    end
    target.vm.box_download_insecure = box[:box]
    target.vm.box = box[:box]
    if box.has_key?(:box_version)
      target.vm.box_version = box[:box_version]
    end

    # issues/49
    target.vm.synced_folder '.', '/vagrant', disabled: true

    # IP
    target.vm.network :private_network, ip: box[:ip]

    # OS specific
    if box[:os] == "windows"
      target.vm.guest = :windows
      target.vm.communicator = "winrm"
      target.vm.provision :shell, :path => "../../../vagrant/Install-
WMF3Hotfix.ps1", privileged: false
      target.vm.provision :shell, :path =>
"../../../vagrant/ConfigureRemotingForAnsible.ps1", privileged: false

      # fix ip for vmware
      if ENV['VAGRANT_DEFAULT_PROVIDER'] == "vmware_desktop"
        target.vm.provision :shell, :path => "../../../vagrant/fix_ip.ps1",
privileged: false, args: box[:ip]
      end

    else
      target.vm.communicator = "ssh"
    end

    if box.has_key?(:forwarded_port)
      # forwarded port explicit
      box[:forwarded_port] do |forwarded_port|
        target.vm.network :forwarded_port, guest: forwarded_port[:guest], host:
forwarded_port[:host], host_ip: "127.0.0.1", id: forwarded_port[:id]
      end
    end
  end
end

```

end

end

end

end

3. Go to your Kali > GOAD folder and edit goad.sh and add elk.yml to ANSIBLE_PLAYBOOKS

```
ANSIBLE_PLAYBOOKS="build.yml ad-servers.yml ad-parent_domain.yml ad-child_domain.yml ad-members.yml ad-trusts.yml ad-data.yml ad-gmsa.yml laps.yml ad-relations.yml adcs.yml ad-acl.yml servers.yml security.yml vulnerabilities.yml reboot.yml elk.yml"
```

4. Run the install again with the following

```
./goad.sh -t install -l GOAD -p vmware -m local -a -r elk.yml
```

5. Do **vagrant up** again

6. Access your ELK at <http://192.168.56.50:5601/>

Have fun!